

## **LAPORAN PRAKTIKUM JOBSHEET 10**



Disusun Oleh:

**HAFIZH ARRASYIID SYAHBANA**

**254107060026**

**SIB 1A**

**PROGRAM STUDI D4 SISTEM INFORMASI  
BISNIS JURUSAN TEKNOLOGI INFORMASI  
POLITEKNIK NEGERI  
MALANG 2026**

### 2.1.1 Verifikasi Hasil Percobaan

```
Mahasiswa10.java src/Pertemuan12

1 package Pertemuan12;    "Pertemuan": Unknown word.
2
3 public class Mahasiswa10 {    "Mahasiswa": Unknown word.
4     String nim;
5     String nama;
6     String kelas;    "kelas": Unknown word.
7     double ipk;
8
9     public Mahasiswa10() {}    "Mahasiswa": Unknown word.
10
11     public Mahasiswa10(String nm, String name, String kls, double ip) {    "Mahasiswa"
12         this.nim = nm;
13         this.nama = name;
14         this.kelas = kls;    "kelas": Unknown word.
15         this.ipk = ip;
16     }
17
18     public void tampilInformasi() {    "tampil": Unknown word.
19         System.out.printf(format: "%-15s %-15s %-5s %.1f%n", nama, nim, kelas, ipk);
20     }
21 }
```

```
NodeMahasiswa10.java src/Pertemuan12 2

1 package Pertemuan12;    "Pertemuan": Unknown word.
2
3 public class NodeMahasiswa10 {    "Mahasiswa": Unknown word.
4     Mahasiswa10 data;    Variable data is never read
5     NodeMahasiswa10 next;    Variable next is never read
6
7     public NodeMahasiswa10(Mahasiswa10 data, NodeMahasiswa10 next) {
8         this.data = data;
9         this.next = next;
10    }
11 }
```

SingleLinkedList10.java src/Pertemuan12

```
1 package Pertemuan12;    "Pertemuan": Unknown word.
2
3 public class SingleLinkedList10 {
4     NodeMahasiswa10 head;    "Mahasiswa": Unknown word.
5     NodeMahasiswa10 tail;    "Mahasiswa": Unknown word.
6
7     boolean isEmpty() {
8         return (head == null);
9     }
10
11     public void print() {
12         if (!isEmpty()) {
13             NodeMahasiswa10 tmp = head;    "Mahasiswa": Unknown word.
14             System.out.println(x: "Isi Linked List:");
15             while (tmp != null) {
16                 tmp.data.tampilInformasi();
17                 tmp = tmp.next;
18             }
19             System.out.println();
20         } else {
21             System.out.println(x: "Linked list kosong");    "kosong": Unkn
22         }
23     }
24
25     public void addFirst(Mahasiswa10 input) {    "Mahasiswa": Unknown word
26         NodeMahasiswa10 ndInput = new NodeMahasiswa10(input, next: null);
27         if (isEmpty()) {
28             head = ndInput;
29             tail = ndInput;
30         } else {
31             ndInput.next = head;
32             head = ndInput;
33         }
34     }
35
36     public void addLast(Mahasiswa10 input) {    "Mahasiswa": Unknown word.
37         NodeMahasiswa10 ndInput = new NodeMahasiswa10(input, next: null);
38         if (isEmpty()) {
39             head = ndInput;
40             tail = ndInput;
41         } else {
42             tail.next = ndInput;
43             tail = ndInput;
44         }
45     }
```

```

47     public void insertAfter(String key, Mahasiswa10 input) {      "Mahasiswa":
48         NodeMahasiswa10 ndInput = new NodeMahasiswa10(input, next: null);
49         NodeMahasiswa10 temp = head;      "Mahasiswa": Unknown word.
50         do {
51             if (temp.data.nama.equalsIgnoreCase(key)) {
52                 ndInput.next = temp.next;
53                 temp.next = ndInput;
54                 if (ndInput.next == null) {
55                     tail = ndInput;
56                 }
57                 break;
58             }
59             temp = temp.next;
60         } while (temp != null);
61     }
62
63     public void insertAt(int index, Mahasiswa10 input) {      "Mahasiswa":
64         if (index < 0) {
65             System.out.println(x: "indeks salah");      "indeks": Unknown wo
66         } else if (index == 0) {
67             addFirst(input);
68         } else {
69             NodeMahasiswa10 temp = head;      "Mahasiswa": Unknown word.
70             for (int i = 0; i < index - 1; i++) {
71                 temp = temp.next;
72             }
73             temp.next = new NodeMahasiswa10(input, temp.next);      "Mahasi
74             if (temp.next.next == null) {
75                 tail = temp.next;
76             }
77         }
78     }
79 }

```

```

SLLMain10.java src/Pertemuan12
1 package Pertemuan12; "Pertemuan": Unknown word.
2
3 public class SLLMain10 {
4     public static void main(String[] args) {
5
6         SingleLinkedList10 sll = new SingleLinkedList10();
7
8         Mahasiswa10 mhs1 = new Mahasiswa10(nm: "22212202", name: "Cintia", kls: "3C", ip: 3.5);
9         Mahasiswa10 mhs2 = new Mahasiswa10(nm: "23212201", name: "Bimon", kls: "2B", ip: 3.8);
10        Mahasiswa10 mhs3 = new Mahasiswa10(nm: "21212203", name: "Dirga", kls: "4D", ip: 3.6);
11        Mahasiswa10 mhs4 = new Mahasiswa10(nm: "24212200", name: "Alvaro", kls: "1A", ip: 4.0);
12
13        sll.print();
14
15        sll.addFirst(mhs3);
16        sll.print();
17
18        sll.addLast(mhs4);
19        sll.print();
20
21        sll.insertAfter(key: "Dirga", mhs1); "Dirga": Unknown word.
22
23        sll.insertAt(index: 2, mhs2);
24        sll.print();
25    }
26 }

```

```

Linked list kosong
Isi Linked List:
Dirga      21212203      4D      3,6

Isi Linked List:
Dirga      21212203      4D      3,6
Alvaro     24212200      1A      4,0

Isi Linked List:
Dirga      21212203      4D      3,6
Cintia     22212202      3C      3,5
Bimon      23212201      2B      3,8
Alvaro     24212200      1A      4,0

```

### 2.1.2 Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan "Linked List Kosong"?

Jawab :

Karena sll.print() dipanggil sebelum ada data yang ditambahkan. head masih null, sehingga isEmpty() mengembalikan true dan program masuk ke blok else yang mencetak "Linked list kosong".

2. Jelaskan kegunaan variable temp secara umum pada setiap method!

Jawab :

temp adalah pointer traversal sementara yang digunakan untuk menelusuri node-node linked list tanpa menggeser head dan tail yang asli. Setiap method menggunakannya sebagai "penunjuk jalan" yang bergerak maju via temp = temp.next. Dilengkapi tabel penjelasan per-method dan ilustrasi kode.

3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawab :

```
public void addFromKeyboard() {
    Scanner sc = new Scanner(System.in);    Resource leak: 'sc' is never closed
    System.out.print(s: "NIM      : "); String nim = sc.nextLine();
    System.out.print(s: "Nama      : "); String nama = sc.nextLine();
    System.out.print(s: "Kelas    : "); String kelas = sc.nextLine();    "Kelas": Unknown
    System.out.print(s: "IPK      : "); double ipk = Double.parseDouble(sc.nextLine());
    addLast(new Mahasiswa10(nim, nama, kelas, ipk));    "Mahasiswa": Unknown word.
    System.out.println(x: "Data berhasil ditambahkan.\n");    "berhasil": Unknown word.
}
```

```
System.out.println(x: "=====");
System.out.println(x: "  Tambah Data Mahasiswa dari Keyboard");    "Tambah": U
System.out.println(x: "=====");

boolean lanjut = true;    "lanjut": Unknown word.
while (lanjut) {    "lanjut": Unknown word.
    System.out.println(x: "\nPilih posisi penambahan:");    "Pilih": Unknown w
    System.out.println(x: "  1. Tambah di depan (addFirst)");    "Tambah": Unk
    System.out.println(x: "  2. Tambah di belakang (addLast)");    "Tambah": U
    System.out.println(x: "  3. Tambah setelah nama tertentu (insertAfter)");
    System.out.println(x: "  4. Tambah pada indeks tertentu (insertAt)");    "
    System.out.println(x: "  0. Selesai");    "Selesai": Unknown word.
    System.out.print(s: "Pilihan: ");    "Pilihan": Unknown word.
    String pilihan = sc.nextLine().trim();    "pilihan": Unknown word.

    if (pilihan.equals(anObject: "0")) {    "pilihan": Unknown word.
        lanjut = false;    The assigned value is never used
        break;
    }

    System.out.println(x: "\n-- Input Data Mahasiswa --");    "Mahasiswa": Unk
    System.out.print(s: "NIM      : "); String nim = sc.nextLine();
    System.out.print(s: "Nama      : "); String nama = sc.nextLine();
    System.out.print(s: "Kelas    : "); String kelas = sc.nextLine();    "Kelas":
    System.out.print(s: "IPK      : ");
    double ipk;
    try { ipk = Double.parseDouble(sc.nextLine()); }
    catch (NumberFormatException e) { ipk = 0.0; }

    Mahasiswa10 baru = new Mahasiswa10(nim, nama, kelas, ipk);    "Mahasiswa":
```

```

switch (pilihan) {    Convert switch to rule switch
    case "1":
        sll.addFirst(baru);    "baru": Unknown word.
        System.out.println(x: "[OK] Data ditambahkan di depan.");    "ditamba
        break;
    case "2":
        sll.addLast(baru);    "baru": Unknown word.
        System.out.println(x: "[OK] Data ditambahkan di belakang.");    "dita
        break;
    case "3":
        System.out.print(s: "Sisipkan setelah nama: ");    "Sisipkan": Unknow
        String key = sc.nextLine();
        sll.insertAfter(key, baru);    "baru": Unknown word.
        System.out.println("[OK] Data disisipkan setelah " + key + ".");    "
        break;
    case "4":
        System.out.print(s: "Masukkan indeks: ");    "Masukkan": Unknown word
        int idx;
        try { idx = Integer.parseInt(sc.nextLine()); }
        catch (NumberFormatException e) { idx = 0; }
        sll.insertAt(idx, baru);    "baru": Unknown word.
        System.out.println("[OK] Data disisipkan pada indeks " + idx + ".");
        break;
    default:
        System.out.println(x: "[!] Pilihan tidak valid.");    "Pilihan": Unkn
}

System.out.println();
sll.print();
}

System.out.println(x: "\n-- Linked List Akhir --");    "Akhir": Unknown word.
sll.print();
sc.close();
}

```

```

Linked list kosong
Isi Linked List:
Dirga      21212203      4D      3,6

Isi Linked List:
Dirga      21212203      4D      3,6
Alvaro     24212200      1A      4,0

Isi Linked List:
Dirga      21212203      4D      3,6
Cintia     22212202      3C      3,5
Bimon      23212201      2B      3,8
Alvaro     24212200      1A      4,0

=====
Tambah Data Mahasiswa dari Keyboard
=====

Pilih posisi penambahan:
1. Tambah di depan (addFirst)
2. Tambah di belakang (addLast)
3. Tambah setelah nama tertentu (insertAfter)
4. Tambah pada indeks tertentu (insertAt)
0. Selesai
Pilihan: 1

-- Input Data Mahasiswa --
NIM : 123
Nama : Jihan
Kelas : 1I
IPK : 4.7
[OK] Data ditambahkan di depan.

Isi Linked List:
Jihan      123          1I      4,7
Dirga      21212203     4D      3,6
Cintia     22212202     3C      3,5
Bimon      23212201     2B      3,8
Alvaro     24212200     1A      4,0

Pilih posisi penambahan:
1. Tambah di depan (addFirst)
2. Tambah di belakang (addLast)
3. Tambah setelah nama tertentu (insertAfter)
4. Tambah pada indeks tertentu (insertAt)
0. Selesai
Pilihan: 2

-- Input Data Mahasiswa --
NIM : 124
Nama : Kaka
Kelas : 4C
IPK : 2.3
[OK] Data ditambahkan di belakang.

Isi Linked List:
Jihan      123          1I      4,7
Dirga      21212203     4D      3,6
Cintia     22212202     3C      3,5
Bimon      23212201     2B      3,8
Alvaro     24212200     1A      4,0
Kaka       124          4C      2,3

Pilih posisi penambahan:
1. Tambah di depan (addFirst)
2. Tambah di belakang (addLast)
3. Tambah setelah nama tertentu (insertAfter)
4. Tambah pada indeks tertentu (insertAt)
0. Selesai
Pilihan: 3

-- Input Data Mahasiswa --
NIM : 125
Nama : Nana
Kelas : 2A
IPK : 4,0
Sisipkan setelah nama: Dirga
[OK] Data disisipkan setelah Dirga.

Isi Linked List:
Jihan      123          1I      4,7
Dirga      21212203     4D      3,6
Nana       125          2A      0,0
Cintia     22212202     3C      3,5
Bimon      23212201     2B      3,8
Alvaro     24212200     1A      4,0
Kaka       124          4C      2,3

```

```

Pilih posisi penambahan:
1. Tambah di depan (addFirst)
2. Tambah di belakang (addLast)
3. Tambah setelah nama tertentu (insertAfter)
4. Tambah pada indeks tertentu (insertAt)
0. Selesai
Pilihan: 4

-- Input Data Mahasiswa --
NIM : 126
Nama : Froen
Kelas : 2H
IPK : 3,5
Masukkan indeks: 3
[OK] Data disisipkan pada indeks 3.

Isi Linked List:
Jihan      123          1I      4,7
Dirga      21212203     4D      3,6
Nana       125          2A      0,0
Froen      126          2H      3,5
Cintia     22212202     3C      3,5
Bimon      23212201     2B      3,8
Alvaro     24212200     1A      4,0
Kaka       124          4C      2,3

Pilih posisi penambahan:
1. Tambah di depan (addFirst)
2. Tambah di belakang (addLast)
3. Tambah setelah nama tertentu (insertAfter)
4. Tambah pada indeks tertentu (insertAt)
0. Selesai
Pilihan: 0

-- Linked List Akhir --
Isi Linked List:
Jihan      123          1I      4,7
Dirga      21212203     4D      3,6
Nana       125          2A      0,0
Froen      126          2H      3,5
Cintia     22212202     3C      3,5
Bimon      23212201     2B      3,8
Alvaro     24212200     1A      4,0
Kaka       124          4C      2,3

```



### 2.2.2 Verifikasi Hasil Percobaan

```
public void getData(int index) {
    NodeMahasiswa10 tmp = head;    "Mahasiswa": Unknown word.
    for (int i = 0; i < index; i++) tmp = tmp.next;
    tmp.data.tampilInformasi();
}

public int indexOf(String key) {
    NodeMahasiswa10 tmp = head;    "Mahasiswa": Unknown word.
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }
    return (tmp == null) ? -1 : index;
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        NodeMahasiswa10 temp = head;    "Mahasiswa": Unknown word.
        while (temp.next != tail) temp = temp.next;
        temp.next = null;
        tail = temp;
    }
}
```

```

public void remove(String key) {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih Kosong, tidak dapat dihapus!");
    } else {
        NodeMahasiswa10 temp = head;    "Mahasiswa": Unknown word.
        while (temp != null) {
            if (temp.data.nama.equalsIgnoreCase(key) && temp == head) {
                this.removeFirst();
                break;
            } else if (temp.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) tail = temp;
                break;
            }
            temp = temp.next;
        }
    }
}

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        NodeMahasiswa10 temp = head;
        for (int i = 0; i < index - 1; i++) temp = temp.next;
        temp.next = temp.next.next;
        if (temp.next == null) tail = temp;
    }
}
}

```

```

System.out.print(s: "data index 1 : \n");
sll.getData(index: 1);
System.out.println();

System.out.println("data mahasiswa an Bimon berada pada index : " + sll.indexOf(key: "bimon"));
System.out.println();

sll.removeFirst();
sll.removeLast();
sll.print();

sll.removeAt(index: 0);
sll.print();

```

```

Isi Linked List:
Dirga      21212203      4D      3,6
Cintia     22212202      3C      3,5
Bimon      23212201      2B      3,8
Alvaro     24212200      1A      4,0

data index 1 :
Cintia     22212202      3C      3,5

data mahasiswa an Bimon berada pada index : 2

Isi Linked List:
Cintia     22212202      3C      3,5
Bimon      23212201      2B      3,8

Isi Linked List:
Bimon      23212201      2B      3,8

```

### 2.2.3 Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!

Jawab :

break digunakan karena dua alasan:

- Menghindari error, setelah node dihapus, temp.next sudah berubah. Jika loop terus berjalan dan mengakses temp = temp.next, bisa terjadi NullPointerException
- Efisiensi, method remove() hanya perlu menghapus satu node pertama yang cocok. Setelah ditemukan dan dihapus, iterasi selanjutnya tidak diperlukan

2. Jelaskan kegunaan kode dibawah pada method remove

```

1  temp.next = temp.next.next;
2  if (temp.next == null) {
3      tail = temp;
4  }

```

Jawab :

Baris 1, temp.next = temp.next.next Memotong node target dari linked list. Node sebelum target (temp) langsung dihubungkan ke node dua langkah ke depan, sehingga node target terlewat dan tidak bisa diakses lagi.

Baris 2 – 4, if (temp.next == null) tail = temp Hanya aktif ketika node yang dihapus adalah node terakhir (tail). Setelah penghapusan, temp.next menjadi null, maka tail harus diperbarui ke temp agar pointer tail tetap valid. Tanpa ini, addLast() berikutnya akan menambah data setelah node yang sudah tidak ada.

### 3 Tugas

```
public class TMahasiswa10 {    "Mahasiswa": Unknown word.
    String nim;
    String nama;
    String kelas;    "kelas": Unknown word.
    double ipk;
    String keperluan;    "keperluan": Unknown word.

    public TMahasiswa10() {    "Mahasiswa": Unknown word.
    }

    public TMahasiswa10(String nim, String nama, String kelas, double ipk, String keperluan) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;    "kelas": Unknown word.
        this.ipk = ipk;
        this.keperluan = keperluan;    "keperluan": Unknown word.
    }

    public void tampilInformasi() {    "tampil": Unknown word.
        System.out.printf(format: " %-12s %-20s %-6s %.2f %-30s%n",
            nim, nama, kelas, ipk, keperluan);    "kelas": Unknown word.
    }
}
```

```
class NodeAntrian10 {    "Antrian": Unknown word.
    TMahasiswa10 data;    "Mahasiswa": Unknown word.
    NodeAntrian10 next;    "Antrian": Unknown word.

    public NodeAntrian10(TMahasiswa10 data, NodeAntrian10 next) {
        this.data = data;
        this.next = next;
    }
}
```

```

public class QueueAntrian10 {    "Antrian": Unknown word.
    private NodeAntrian10 head;    "Antrian": Unknown word.
    private NodeAntrian10 tail;    "Antrian": Unknown word.
    private int size;
    private final int MAX_SIZE;

    public QueueAntrian10(int maxSize) {    "Antrian": Unknown word.
        this.head = null;
        this.tail = null;
        this.size = 0;
        this.MAX_SIZE = maxSize;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public boolean isFull() {
        return size >= MAX_SIZE;
    }

    public void clear() {
        head = null;
        tail = null;
        size = 0;
        System.out.println(x: "    [INFO] Antrian berhasil dikosongkan.");    "Antrian": Unknown word.
    }

    public void enqueue(TMahasiswa10 mhs) {    "Mahasiswa": Unknown word.
        if (isFull()) {
            System.out.println(x: "    [GAGAL] Antrian penuh! Tidak dapat menambah data.");
            return;
        }
        NodeAntrian10 node = new NodeAntrian10(mhs, next: null);    "Antrian": Unknown word.
        if (isEmpty()) {
            head = node;
            tail = node;
        } else {
            tail.next = node;
            tail = node;
        }
        size++;
        System.out.printf(format: "    [OK] %s berhasil masuk antrian (nomor antrian: %d).%n",
            mhs.nama, size);
    }
}

```

```

public TMahasiswa10 dequeue() {    "Mahasiswa": Unknown word.
    if (isEmpty()) {
        System.out.println(x: "    [GAGAL] Antrian kosong! Tidak ada yang bisa dipanggil.");
        return null;
    }
    TMahasiswa10 dilayani = head.data;    "Mahasiswa": Unknown word.
    head = head.next;
    if (head == null) tail = null;
    size--;
    return dilayani;    "dilayani": Unknown word.
}

public void tampilTerdepan() {    "tampil": Unknown word.
    if (isEmpty()) {
        System.out.println(x: "    [INFO] Antrian kosong.");    "Antrian": Unknown word.
    } else {
        System.out.println(x: "    Antrian terdepan:");    "Antrian": Unknown word.
        System.out.printf(format: "    %-12s %-20s %-6s %-5s %-30s\n",
            ...args: "NIM", "Nama", "Kelas", "IPK", "Keperluan");    "Kelas": Unknown word
        System.out.println("    " + "-".repeat(count: 78));
        head.data.tampilInformasi();
    }
}

public void tampilTerakhir() {    "tampil": Unknown word.
    if (isEmpty()) {
        System.out.println(x: "    [INFO] Antrian kosong.");    "Antrian": Unknown word.
    } else {
        System.out.println(x: "    Antrian paling akhir:");    "Antrian": Unknown word.
        System.out.printf(format: "    %-12s %-20s %-6s %-5s %-30s\n",
            ...args: "NIM", "Nama", "Kelas", "IPK", "Keperluan");    "Kelas": Unknown word
        System.out.println("    " + "-".repeat(count: 78));
        tail.data.tampilInformasi();
    }
}

public int jumlahAntrian() {    "jumlah": Unknown word.
    return size;
}

public void tampilAntrian() {    "tampil": Unknown word.
    if (isEmpty()) {
        System.out.println(x: "    [INFO] Antrian kosong.");    "Antrian": Unknown word.
        return;
    }
    System.out.println(x: "    Daftar Antrian Layanan Unit Kemahasiswaan:");    "Daftar": Un
    System.out.printf(format: "    %-4s %-12s %-20s %-6s %-5s %-30s\n",
        ...args: "No.", "NIM", "Nama", "Kelas", "IPK", "Keperluan");    "Kelas": Unknown wo
    System.out.println("    " + "-".repeat(count: 82));
    NodeAntrian10 tmp = head;    "Antrian": Unknown word.
    int no = 1;
    while (tmp != null) {
        System.out.printf(format: "    %-4d", no++);
        tmp.data.tampilInformasi();
        tmp = tmp.next;
    }
    System.out.println("    " + "-".repeat(count: 82));
    System.out.printf(format: "    Total mahasiswa mengantri: %d mahasiswa\n", size);    "ma
}

```

```

import java.util.Scanner;

public class AntrianMain10 {    "Antrian": Unknown word.
    static Scanner sc = new Scanner(System.in);
    static QueueAntrian10 antrian = new QueueAntrian10(maxSize: 10);    "Antrian": Unkn

Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        System.out.println(x: " ");
        System.out.println(x: " SISTEM ANTRIAN LAYANAN UNIT KEMAHASISWAAN ");
        System.out.println(x: " ");

        boolean running = true;
        while (running) {
            tampilMenu();    "tampil": Unknown word.
            System.out.print(s: "Pilih menu: ");    "Pilih": Unknown word.
            String pilihan = sc.nextLine().trim();    "pilihan": Unknown word.

            System.out.println();
            switch (pilihan) {    "pilihan": Unknown word.
                case "1" -> daftarMahasiswa();    "daftar": Unknown word.
                case "2" -> panggilAntrian();    "panggil": Unknown word.
                case "3" -> antrian.tampilAntrian();    "antrian": Unknown word.
                case "4" -> antrian.tampilTerdepan();    "antrian": Unknown word.
                case "5" -> antrian.tampilTerakhir();    "antrian": Unknown word.
                case "6" -> System.out.println(" Jumlah mahasiswa mengantri: "    "Ju
                    + antrian.jumlahAntrian() + " orang");    "antrian": Unknown w
                case "7" -> cekStatus();
                case "8" -> {
                    System.out.print(s: " Yakin mengosongkan antrian? (y/n): ");    "Y
                    if (sc.nextLine().trim().equalsIgnoreCase(anotherString: "y"))
                        antrian.clear();    "antrian": Unknown word.
                }
                case "0" -> {
                    System.out.println(x: " Sistem antrian ditutup. Terima kasih!");
                    running = false;
                }
                default -> System.out.println(x: " [!] Pilihan tidak valid.");    "Pi
            }
            System.out.println();
        }
    }
}

```

```

static void tampilMenu() {    "tampil": Unknown word.
    System.out.println(x: "-----");
    System.out.println(x: " [1] Daftar Antrian (Enqueue)");    "Daftar": Unknown word.
    System.out.println(x: " [2] Panggil / Layani Antrian (Dequeue)");    "Panggil": Unknown
    System.out.println(x: " [3] Tampilkan Semua Antrian");    "Tampilkan": Unknown word.
    System.out.println(x: " [4] Tampilkan Antrian Terdepan");    "Tampilkan": Unknown word
    System.out.println(x: " [5] Tampilkan Antrian Paling Akhir");    "Tampilkan": Unknown
    System.out.println(x: " [6] Jumlah Mahasiswa Mengantri");    "Jumlah": Unknown word.
    System.out.println(x: " [7] Cek Status Antrian (Kosong / Penuh)");    "Antrian": Unkn
    System.out.println(x: " [8] Kosongkan Antrian");    "Kosongkan": Unknown word.
    System.out.println(x: " [0] Keluar");    "Keluar": Unknown word.
    System.out.println(x: "-----");
}

static void daftarMahasiswa() {    "daftar": Unknown word.
    if (antrian.isFull()) {    "antrian": Unknown word.
        System.out.println(x: " [GAGAL] Antrian penuh!");    "GAGAL": Unknown word.
        return;
    }
    System.out.println(x: " -- Pendaftaran Antrian --");    "Pendaftaran": Unknown word.
    System.out.print(s: " NIM      : ");
    String nim = sc.nextLine();
    System.out.print(s: " Nama      : ");
    String nama = sc.nextLine();
    System.out.print(s: " Kelas      : ");    "Kelas": Unknown word.
    String kls = sc.nextLine();
    System.out.print(s: " IPK      : ");
    double ipk;
    try {
        ipk = Double.parseDouble(sc.nextLine());
    } catch (NumberFormatException e) {
        ipk = 0.0;
    }
    System.out.print(s: " Keperluan : ");    "Keperluan": Unknown word.
    String kep = sc.nextLine();

    TMahasiswa10 mhs = new TMahasiswa10(nim, nama, kls, ipk, kep);    "Mahasiswa": Unknown
    antrian.enqueue(mhs);    "antrian": Unknown word.
}

static void panggilAntrian() {    "panggil": Unknown word.
    System.out.println(x: " -- Memanggil Antrian Berikutnya --");    "Memanggil": Unknown
    TMahasiswa10 dilayani = antrian.dequeue();    "Mahasiswa": Unknown word.
    if (dilayani != null) {    "dilayani": Unknown word.
        System.out.println(x: " ✓ Mahasiswa yang dipanggil untuk dilayani:");    "Mahasis
        System.out.printf(format: "      Nama      : %s\n", dilayani.nama);    "dilayani": U
        System.out.printf(format: "      NIM      : %s\n", dilayani.nim);    "dilayani": Un
        System.out.printf(format: "      Kelas     : %s\n", dilayani.kelas);    "Kelas": Unk
        System.out.printf(format: "      IPK      : %.2f\n", dilayani.ipk);    "dilayani":
        System.out.printf(format: "      Keperluan : %s\n", dilayani.keperluan);    "Keperlu
        System.out.printf(format: " Sisa antrian: %d orang\n", antrian.jumlahAntrian());
    }
}

static void cekStatus() {
    System.out.println(x: " Status antrian:");    "antrian": Unknown word.
    System.out.println(" Kosong : " + (antrian.isEmpty() ? "YA" : "TIDAK"));    "Kosong
    System.out.println(" Penuh  : " + (antrian.isFull() ? "YA" : "TIDAK"));
    System.out.println(" Jumlah : " + antrian.jumlahAntrian() + " orang");
}

```



## SISTEM ANTRIAN LAYANAN UNIT KEMAHASISWAAN

- [1] Daftar Antrian (Enqueue)
- [2] Panggil / Layani Antrian (Dequeue)
- [3] Tampilkan Semua Antrian
- [4] Tampilkan Antrian Terdepan
- [5] Tampilkan Antrian Paling Akhir
- [6] Jumlah Mahasiswa Mengantri
- [7] Cek Status Antrian (Kosong / Penuh)
- [8] Kosongkan Antrian
- [0] Keluar

Pilih menu: 1

-- Pendaftaran Antrian --

NIM : 123  
Nama : Nanas  
Kelas : 1A  
IPK : 2.4  
Keperluan : Ngaji  
[OK] Nanas berhasil masuk antrian (nomor antrian: 1).

- [1] Daftar Antrian (Enqueue)
- [2] Panggil / Layani Antrian (Dequeue)
- [3] Tampilkan Semua Antrian
- [4] Tampilkan Antrian Terdepan
- [5] Tampilkan Antrian Paling Akhir
- [6] Jumlah Mahasiswa Mengantri
- [7] Cek Status Antrian (Kosong / Penuh)
- [8] Kosongkan Antrian
- [0] Keluar

Pilih menu: 1

-- Pendaftaran Antrian --

NIM : 124  
Nama : Rafa  
Kelas : 2B  
IPK : 3.4  
Keperluan : Apa aja  
[OK] Rafa berhasil masuk antrian (nomor antrian: 2).

- [1] Daftar Antrian (Enqueue)
- [2] Panggil / Layani Antrian (Dequeue)
- [3] Tampilkan Semua Antrian
- [4] Tampilkan Antrian Terdepan
- [5] Tampilkan Antrian Paling Akhir
- [6] Jumlah Mahasiswa Mengantri
- [7] Cek Status Antrian (Kosong / Penuh)
- [8] Kosongkan Antrian
- [0] Keluar

Pilih menu: 2

-- Memanggil Antrian Berikutnya --

✓ Mahasiswa yang dipanggil untuk dilayani:  
Nama : Nanas  
NIM : 123  
Kelas : 1A  
IPK : 2,40  
Keperluan : Ngaji  
Sisa antrian: 1 orang

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 3

Daftar Antrian Layanan Unit Kemahasiswaan:

| No. | NIM | Nama | Kelas | IPK  | Keperluan |
|-----|-----|------|-------|------|-----------|
| 1   | 124 | Rafa | 2B    | 3,40 | Apa aja   |

---

Total mahasiswa mengantri: 1 mahasiswa

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 1

-- Pendaftaran Antrian --

NIM : 125  
Nama : Refi  
Kelas : 1C  
IPK : 2.3  
Keperluan : Ada  
[OK] Refi berhasil masuk antrian (nomor antrian: 2).

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 4

Antrian terdepan:

| NIM | Nama | Kelas | IPK  | Keperluan |
|-----|------|-------|------|-----------|
| 124 | Rafa | 2B    | 3,40 | Apa aja   |

---

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 5

Antrian paling akhir:

| NIM | Nama | Kelas | IPK  | Keperluan |
|-----|------|-------|------|-----------|
| 125 | Refi | 1C    | 2,30 | Ada       |

---

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 6

Jumlah mahasiswa mengantri: 2 orang

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 7

Status antrian:

Kosong : TIDAK  
Penuh : TIDAK  
Jumlah : 2 orang

- 
- [1] Daftar Antrian (Enqueue)
  - [2] Panggil / Layani Antrian (Dequeue)
  - [3] Tampilkan Semua Antrian
  - [4] Tampilkan Antrian Terdepan
  - [5] Tampilkan Antrian Paling Akhir
  - [6] Jumlah Mahasiswa Mengantri
  - [7] Cek Status Antrian (Kosong / Penuh)
  - [8] Kosongkan Antrian
  - [0] Keluar
- 

Pilih menu: 8

Yakin mengosongkan antrian? (y/n): y  
[INFO] Antrian berhasil dikosongkan.

---

```
[1] Daftar Antrian (Enqueue)
[2] Panggil / Layani Antrian (Dequeue)
[3] Tampilkan Semua Antrian
[4] Tampilkan Antrian Terdepan
[5] Tampilkan Antrian Paling Akhir
[6] Jumlah Mahasiswa Mengantri
[7] Cek Status Antrian (Kosong / Penuh)
[8] Kosongkan Antrian
[0] Keluar
```

---

Pilih menu: 3

[INFO] Antrian kosong.

---

```
[1] Daftar Antrian (Enqueue)
[2] Panggil / Layani Antrian (Dequeue)
[3] Tampilkan Semua Antrian
[4] Tampilkan Antrian Terdepan
[5] Tampilkan Antrian Paling Akhir
[6] Jumlah Mahasiswa Mengantri
[7] Cek Status Antrian (Kosong / Penuh)
[8] Kosongkan Antrian
[0] Keluar
```

---

Pilih menu: 0

Sistem antrian ditutup. Terima kasih!

LINK GITHUB : <https://github.com/nNasq/Jobsheet-11---Linked-List>